SECURITY · PENTEST 2026-09

Pentest triage

Haboob's web penetration test of register.tourise.com for the Saudi Tourism Authority, checked finding by finding against the code.

Classification: **CONFIDENTIAL / TLP:AMBER** · Checked: **2026-10-13** · Code: **backend 0ef8445, frontend bd4fa79, admin c21725d (dev = main)**

1. In short

Haboob tested the public registration site from outside, with no login, on 14 and 15 September 2026, and reported five findings, all **Medium**. Each one was checked against the code on 13 October 2026. None is a false alarm, but two are not quite what the report says:

- **M-002 is a production setting, not a code bug.** The stack traces in the report only appear when the backend runs with `APP_DEBUG=true`. The code already hides them when debug is off. Debug cannot simply be switched off, though: today's error handler would then turn every validation, login and permission error into a 500 (section 4.2).
- **M-004 is real, but not in the way the report shows.** Rewriting the browser's response only moves the form on to the next step. Registration checks the code again on the server. The real gap is that the code is not tied to the email address, so a person can confirm their own address and register someone else's.

Most urgent, updated 2026-10-13: the production backend's `.env` now has `APP_DEBUG=false` (seen by the owner on the server), so the stack traces should be gone. That makes the error handler problem live: with debug off it answers a missing login, a permission denial and a validation error as a 500, on production today (section 4.2). **Fixed the same day:** Task 041 B9, backend `c8a1885`, on `dev` and `main`; it reaches production with the next backend deploy.

2. Findings at a glance

ID	Finding	Their rating	Verdict	Track	Status
M-001	Outdated Next.js	MEDIUM	Valid. Both Next apps on 15.5.19	CODE	OPEN
M-002	Improper error handling	MEDIUM	Valid. Debug now off; handler fixed, to deploy	OPS + CODE	FIXED
M-003	Database column names disclosed	MEDIUM	Valid, and wider than reported	CODE	OPEN

ID	Finding	Their rating	Verdict	Track	Status
M-004	OTP bypass	MEDIUM	Partly valid. Different weakness, same fix area	CODE	OPEN
M-005	User enumeration	MEDIUM	Valid. A second route does the same	DECISION	OPEN

Tracks. Code: a repository change. Ops: an environment setting on the server, no repository change. Decision: needs the owner's call before work starts.

Task 041. Three of these were already on the security wave 1 list (Task 041), opened 2026-09-21 with nothing closed yet: **B11** is M-003 and M-005's route, **B12** is M-004, **B9** is the error handler that M-002 depends on. Haboob found them independently.

3. What was tested

Item	Detail
Assessor	Haboob, for the Saudi Tourism Authority (STA)
Report	"Web Penetration Testing Report for STA - Tourise Register - 72", version 1.0, approved 15/09/2026
Window	14 to 15 September 2026
Type	Black box, external. Denial of service and social engineering out of scope
Targets	`register.tourise.com/en/`, `/en/join/vip`, `/en/join/general-registration`, `/en/join/request-an-invitation`
Also reached	`api-register.tourise.com` (the backend the forms call), and `/en/join/media`
Not tested	The admin panel, the mobile API, anything behind a login

4. Finding by finding

4.1 M-001 · Outdated Next.js

Reported. The public site's JavaScript shows Next.js 15.5.19, which has eight known CVEs (CVE-2026-64641 to CVE-2026-64649).

Checked. Confirmed: `next` 15.5.19 in `tourise-frontend/yarn.lock`, and the admin is on the same version (not in scope, same fix). The eight CVEs are the July 2026 Next.js security release; NVD lists CVE-2026-64645 as affecting up to 15.5.20 and fixed in 15.5.21. The newest 15.x on npm today is 15.5.27, which also carries the September 2026 fixes.

Fix. Raise `next` to 15.5.27 in frontend and admin. That stays inside Next 15, which the project rules allow without a dedicated branch. Gate: `yarn type-check` and `yarn build` in both apps. Before closing, confirm the eight IDs against Next.js's own GitHub advisories.

4.2 M-002 · Improper error handling

Reported. Three backend addresses answered with Laravel's debug output: the exception class, server file paths under `/var/www/tourise-backend`, line numbers and stack traces. Two of them were SQL errors that also named the database's internal address, port and name, and one printed the whole insert statement with the submitted form's personal data.

Checked. The code is right; the server setting was wrong. `app/Exceptions/Handler.php:83` returns a bare "Internal server error." for API errors when `config('app.debug')` is false, and `config/app.php:76` defaults it to false. The output in the screenshots only exists with `APP_DEBUG=true`, so production ran with debug on during the test. The three errors behind it:

- `POST /api/guests/check-unique` with a made-up column name, a SQL error. Goes away with M-003.
- `POST /api/guests` with a 36-character value in `country_of_residence`, "Data too long for column". `GuestsController::store` saves that field with no length or format check (`GuestsController.php:649`).
- `GET /api/upload`, a 405 for the wrong method. Harmless once debug is off.

> **Do not switch debug off on its own.** With debug off, the same handler answers every error it does not name (validation 422, login 401, permission 403, wrong method 405, any `abort()`) as a 500 "Internal server error." The public forms, the admin and the mobile app would get a 500 where they expect the real status. Task 041 **B9** rewrites the handler to keep those statuses; it has to ship first, together with **L9** (the mobile team confirms their app handles a 401). pep-v2 hit the same thing: 48 tests failed once debug was off.

Fix, in this order. (1) Code: Task 041 B9, the handler keeps 4xx statuses and hides only real 500s. (2) Code: validate `country_of_residence` and the other free-text fields that land in fixed-length columns. (3) Ops: `APP_DEBUG=false` on the production backend, then `php artisan config:cache`; confirm `/_ignition/health-check` returns 404 and that production was installed with `composer install --no-dev` (Task 041 devops list). The repository's `.env.example` ships `APP_DEBUG=true`, so a server built by copying it starts in debug.

Reported before the test too. The same leak was posted in the team's WhatsApp group on 2026-09-13, the day before Haboob started: a `GET` on `/api/guests/check-unique` answered its 405 with the exception class, file paths and a full trace. That message asked for two more checks:

- **No debug tooling reachable.** The backend has no Telescope and no log viewer; Ignition is in `require-dev` only, so a production install with `composer install --no-dev` does not have it. Confirm `/_ignition/health-check` answers 404.
- **CORS limited to the site.** `config/cors.php` reads the allowed origins from `CORS_ALLOWED_ORIGINS` and **falls back to `*` (any site) when it is not set**; `.env.example` does not mention it. Haboob's captured responses carry `Access-Control-Allow-Origin: https://register.tourise.com`, so production had it set on 14 September. Confirm the production `.env` lists only the public site and the admin.

Check now. `GET https://api-register.tourise.com/api/upload` should answer 405 with only a message. If it shows `"file"` and `"trace"`, debug is still on.

> **Update 2026-10-13: debug is off on production.** The owner found `APP_DEBUG=false` in the production backend's `.env` (when it was changed is not recorded). Two consequences:
>
> **The running app may not have read it yet.** If the config is cached, the `.env` is ignored until the cache is rebuilt. On the server, `php artisan config:show app.debug` must print `false`; if not, run `php artisan config:cache`.
>
> **The status codes are now wrong on production.** The full backend suite run locally with `APP_DEBUG=false`: **94 of 1,153 tests fail**, every one a status check, a 401, 403 or 422 that came back as 500 (43 expected 401, 34 expected 403, 13 expected 422, 4 more through status lists). Among them `MobileAuthTest` and the partner APIs (`PartnerInvitationRequestApiTest`, `PartnerNewsletterApiTest`), so a partner's server sending a bad request gets a 500 with no reason. Anything that reacts to the status, such as the re-login on 401 that the mobile contract documents, gets a 500 instead, and real server errors are hidden among them. B9 is no longer a step before the switch; it is the repair after it.
>
> **Fixed 2026-10-13:** backend `c8a1885` (branch `fix/api-error-status-codes`, PR #64 to `dev`, PR #65 to `main`), ported from pep-v2. A missing or expired token answers 401, a thrown validation error 422 with its errors, any other 4xx keeps its status and message; a 5xx and anything unexpected stay a bare "Internal server error.". `Authenticate::redirectTo` no longer names a `login` route that does not exist. New `ApiErrorHandlingTest` (10 tests, debug forced off, 7 fail on the old code); the suite passes 1,163 of 1,163 with debug on and off. Not deployed yet.

4.3 M-003 · Database column names disclosed

Reported. Sending `check-unique` a `type` of any word made the backend run a query on a column of that name. Unknown names returned SQL errors, so trying 2,887 names revealed which columns the guests table has.

Checked. Confirmed, and it reaches further than reported. `GuestsController::checkUnique` (`GuestsController.php:2033`) runs `Guest::where($request['type'], $request['value'])`: the caller picks the column. The public route is `routes/api.php:161`, with no login and no reCAPTCHA.

- With debug off the column names stop leaking, but the route still answers yes or no for **any** column that exists: national ID, passport number, phone. One value at a time, for anyone.
- With `mode=edit` and a guest's id it returns **the whole guest record**. Haboob did not find this; it needs an id they never had.
- Since the test it has lost its rate limit. Haboob's screenshots show a limit of 30 a minute; after the 2026-09-30 limiter change this route sits in a group with none.

The code came with the project: it is in the first backend commit, `947c658` (2026-06-21, from the pif-directors-gathering baseline), unchanged since. Sibling ALT projects carry the same code, which is why Task 041 lists it as **B11**.

Fix. Every caller sends `type: 'email'`: the ten public join forms (`tourise-frontend/utils/unique-attribute.ts`) and the admin's guest forms (`tourise-admin/utils/unique-attribute.ts`), which only

read success or failure, never the record. The same method also serves the logged-in admin route (`routes/api.php:388`). Accept `email` only, validate it as an email, and never return the guest record. Tests for an unknown type, a non-email type and `mode=edit`.

4.4 M-004 · OTP bypass

Reported. On the media form, Haboob typed a wrong email code, caught the server's answer (404, "wrong token") and rewrote it to 200 "Success". The form accepted it and moved to the next step.

Checked. Partly valid. `POST /api/guests/email-confirmation` only decides whether the form moves on. The real check happens when the form is submitted: `GuestsController::store` refuses the registration unless the code exists in `email_verifications` (`GuestsController.php:601`). Rewriting the response therefore does not register anyone, and the report does not show a registration completing; its last screenshot is the next form step.

The server check is weaker than it should be, in a different way: it looks the code up **by itself**, not together with the email being registered (Task 041 **B12**). So a person can confirm their own address, then submit a registration under **someone else's** email with that code. The confirmation endpoint has the same gap (`AuthController.php:572`). Email codes also never expire, unlike phone codes (`phone_otp_ttl_minutes`). The request-an-invitation form already does it right: `InvitationRequestsController.php:466` matches code **and** email.

Haboob's other suggestions are already met: codes are eight random letters and digits (`EmailOtpSender`), far too many to guess, a used code is deleted after registration, and the route is rate limited (600 a minute per IP, `sensitive-api`).

Fix. Match the code together with the trimmed, lower-cased email in `store` and in `emailConfirmation`, as the invitation request already does. Optionally an expiry like the phone codes. Tests: a code for one address refused for another.

4.5 M-005 · User enumeration

Reported. The registration form says "This email is already used", so anyone can learn whether an address is registered.

Checked. Confirmed: `POST /api/guests/check-unique` answers 200 for a free email and 422 for a registered one, with no login and no reCAPTCHA (and, as in 4.3, no rate limit now).

A second route does the same and was not tested, because it did not exist yet: `POST /api/invitation-requests/check-unique` (`routes/api.php:259`), added on 2026-09-30 (backend `c212d74`, frontend `4d6dc7c`), answers "already registered" or "already requested" for any email. Fixing only the first leaves the finding open, and a retest would most likely report the second.

Decision needed. Telling a returning guest "you are already registered" is a product feature, so this is the owner's call:

Option	What changes	Cost
A. Remove the live lookups	The forms stop asking while the person types; the duplicate is still refused on submit, which sits behind reCAPTCHA	The person learns at the end of the form, not at the email field

Option	What changes	Cost
B. Keep them behind reCAPTCHA	Each lookup needs a reCAPTCHA token	Slows a script, still answers it
C. Haboob's generic message	The form never says whether an address is registered	A real guest is not told why they get no second confirmation

A was recommended for the invitation-request route on 2026-09-30. If the choice is A or C, both routes change together.

5. Order of work

1. **Done 2026-10-13:** `APP_DEBUG=false` in the production `.env`. Still to confirm on the server: `php artisan config:show app.debug` prints `false`, `_ignition` returns 404, installed with `--no-dev`.
2. **Code, backend:** B9 first, done 2026-10-13 (backend `c8a1885`, on `dev` and `main`). Then B11 (4.3), B12 (4.4), the `country_of_residence` check (4.2). Each with tests, in `composer qa`.
3. **Mobile:** ask the mobile team whether the app handles a 401 (Task 041 L9). With debug off it gets a 500 today, so B9 makes this better, not worse.
4. **Ops:** deploy the backend from `main` (no migration), then check `/api/admin/roles/select` answers 401 and `/api/upload` 405 with no `file` or `trace`.
5. **Code, frontend and admin:** Next.js 15.5.27 (4.1).
6. **Owner:** choose A, B or C (4.5), then the code for it.
7. **Retest:** ask Haboob to retest the five findings, or run their requests ourselves on staging first.

6. Sources

- The report itself: kept outside the repository, see the pentest README.
- NVD, CVE-2026-64645: affected versions and the 15.5.21 fix.
- Netlify, Next.js security release, July 2026 and September 2026.
- Task 041 items B9, B11, B12, L9: ITEMS.md.